

Disha Meghnani – dxm478@student.bham.ac.uk

Priyanka Pawar- ppx459@student.bham.ac.uk

Prutha Mane – pxm491@student.bham.ac.uk

SOFTWARE SECURITY AND FUZZING

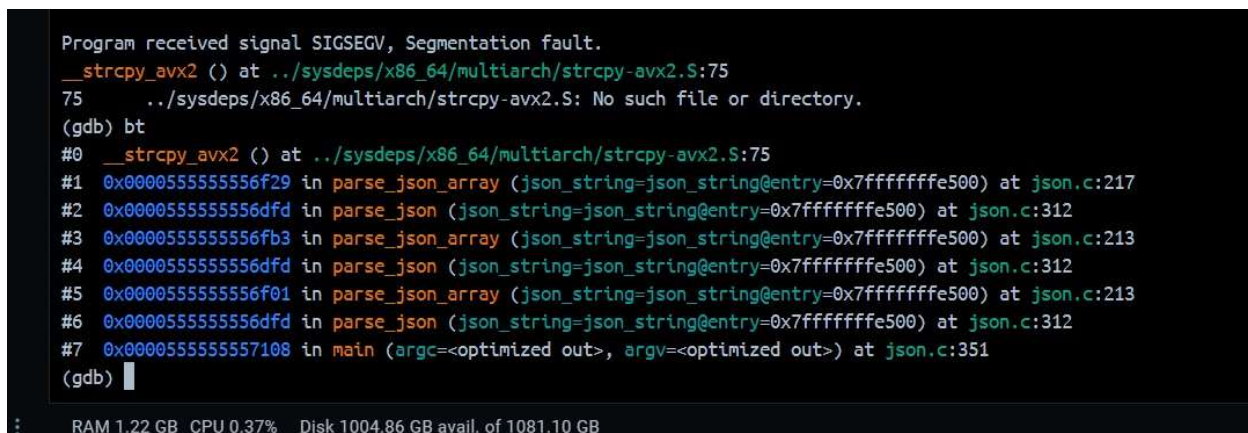
ASSIGNMENT 1- GROUP 03

We automated the AFL fuzzing process and incorporated the necessary dependencies into the **Dockerfile**. Within the fuzz/in/ directory, we created a seed file to serve as input for the fuzzer. Using this setup, we built the AFL fuzzing environment and launched it inside the Docker container. Once the container was up and running, AFL began executing and detecting crashes in the target file, **json.c**.



american fuzzy lop ++4.32a [default] (/target/json_fuzz) [explore]	
process timing	overall results
run time : 0 days, 0 hrs, 2 min, 49 sec	cycles done : 1
last new find : 0 days, 0 hrs, 0 min, 14 sec	corpus count : 76
last saved crash : 0 days, 0 hrs, 0 min, 10 sec	saved crashes : 1
last saved hang : none seen yet	saved hangs : 0
cycle progress	map coverage
now processing : 38.6 (50.0%)	map density : 15.52% / 65.52%
runs timed out : 0 (0.00%)	count coverage : 4.26 bits/tuple
stage progress	findings in depth
now trying : havoc	favorited items : 16 (21.05%)
stage execs : 37/100 (37.00%)	new edges on : 22 (28.95%)
total execs : 40.9k	total crashes : 1 (1 saved)
exec speed : 254.1/sec	total tmouts : 1 (0 saved)
fuzzing strategy yields	item geometry
bit flips : 0/128, 0/127, 0/125	levels : 7
byte flips : 0/16, 0/15, 0/13	pending : 25
arithmetics : 0/1106, 0/1960, 0/1680	pend fav : 0
known ints : 0/140, 0/560, 0/718	own finds : 75
dictionary : 0/0, 0/0, 0/0, 0/0	imported : 0
havoc/splice : 75/39.8k, 0/0	stability : 100.00%
py/custom/rq : unused, unused, unused, unused	
trim/eff : 44.06%/242, 93.75%	[cpu000: 37%]

The crashes identified during the fuzzing process were stored in the out folder. To further analyze them, we accessed the crash folder using Bash and examined the issue with **GDB (GNU Debugger)**. Upon examination, we discovered that the crash was caused by a **segmentation fault**, indicating a critical memory access error in the program.



```
Program received signal SIGSEGV, Segmentation fault.
__strcpy_avx2 () at ../sysdeps/x86_64/multiarch/strcpy-avx2.S:75
75      ../sysdeps/x86_64/multiarch/strcpy-avx2.S: No such file or directory.
(gdb) bt
#0  __strcpy_avx2 () at ../sysdeps/x86_64/multiarch/strcpy-avx2.S:75
#1  0x00005555555556f29 in parse_json_array (json_string=json_string@entry=0x7fffffff500) at json.c:217
#2  0x00005555555556dfd in parse_json (json_string=json_string@entry=0x7fffffff500) at json.c:312
#3  0x00005555555556fb3 in parse_json_array (json_string=json_string@entry=0x7fffffff500) at json.c:213
#4  0x00005555555556dfd in parse_json (json_string=json_string@entry=0x7fffffff500) at json.c:312
#5  0x00005555555556f01 in parse_json_array (json_string=json_string@entry=0x7fffffff500) at json.c:213
#6  0x00005555555556dfd in parse_json (json_string=json_string@entry=0x7fffffff500) at json.c:312
#7  0x00005555555557108 in main (argc=<optimized out>, argv=<optimized out>) at json.c:351
(gdb) |
```

: RAM 1.22 GB CPU 0.37% Disk 1004.86 GB avail. of 1081.10 GB

We took the target file, json.c, and successfully fixed the crash. To document the changes, we created a patch using the diff command and saved it in a file named patch.patch. This patch captures the modifications made to resolve the crash.

```
root@2d4df405235a:/workspace# ls
Dockerfile  in  json.c  out  run.sh
root@2d4df405235a:/workspace# nano json_patched.c
root@2d4df405235a:/workspace# ls
Dockerfile  in  json.c  json_patched.c  out  run.sh
root@2d4df405235a:/workspace# diff -u json.c json_patched.c > patch.patch
root@2d4df405235a:/workspace# ls
Dockerfile  in  json.c  json_patched.c  out  patch.patch  run.sh
root@2d4df405235a:/workspace#
```

To automate the process, we created a **Dockerfile** that sets up the environment, applies the patch, and rebuilds the program. Additionally, a launcher script, run.sh, was developed to automate the entire patching and testing process. This automation ensures consistent, reproducible application of the fix.

```
view a summary of image vulnerabilities and recommendations /docker scout quickview
PS D:\cyber\SSHS\assignment1\assignment1\fixed> docker run -v ./workspace -it sshs_asg1
Error: JSON parsing encountered an issue!
PS D:\cyber\SSHS\assignment1\assignment1\fixed>
```

We take plagiarism seriously. We agree that:

1. The submission is our group's own work and that we have not worked with others in preparing this assignment.
2. Our submitted solutions and report were written by us and in our own words.
3. The work is not copied from any other person's work (published or unpublished), web site, book or other source, and has not previously been submitted for assessment either at the University of Birmingham or elsewhere.
4. We have not asked, or paid, others to prepare any part of this work.